



2950 Niles Road, St. Joseph, MI 49085-9659, USA
269.429.0300 fax 269.429.3852 hq@asabe.org www.asabe.org

An ASABE Meeting Presentation

DOI: <https://doi.org/10.13031/aim.201900487>

Paper Number: 1900487

High-Throughput Image Analysis Framework for Fruit Detection, Localization and Measurement from Video Streams

Yi-Hsuan Huang, Ta-Te Lin

Department of Bio-Industrial Mechatronics Engineering, National Taiwan University

No. 1, Sec. 4, Roosevelt Rd., Taipei 10617, Taiwan, R.O.C.

**Written for presentation at the
2019 ASABE Annual International Meeting**

**Sponsored by ASABE
Boston, Massachusetts
July 7–10, 2019**

ABSTRACT. Food crises and security issues are getting worse due to climate change and population growth. One of the solutions being sought is the use of efficient breeding systems, which requires accurate and detailed phenotyping of fruits and plants. However, traditional phenotyping methods are time consuming, labor intensive and prone to human error. Therefore, measuring the morphological and physiological parameters of fruits automatically is highly recommended. In this work, a high-throughput technique for fruits detection, localization and measurement from video streams using computer vision and deep neural networks is proposed. In contrast with other works that were developed for single type of fruits, a versatile method is proposed herein that can be applied for different types of fruits using a vision system to scan through plants row by row in a greenhouse. A real-time object detection algorithm using YOLOv2, a deep neural network-based detector, is used for fruit detection and localization on video frames with a hit rate of 84.98%. An individual fruit tracking algorithm is applied throughout the video stream to perform tracking of multiple fruits. The online tracking algorithm includes feature matching, optical flow and projective transformation optimized by occlusion handling techniques such as by applying threshold indices and denoising. On the other hand, the offline tracking algorithm uses a voting method to reduce the false alarms caused by the object detector. Finally, phenotyping information such as fruit counts, ripening stage, fruit size, and 2D spatial distribution maps were obtained. The proposed framework has demonstrated its efficacy in obtaining satisfactory phenotyping information that is useful for production management as well as its potential utilization in robotic operations.

Keywords. Automation, Computer vision, Object detection, Object tracking, Phenotyping

Introduction

In recent years, food crises and insecurity issues have been getting worse. The 2018 Global Report on Food Crises reports on a severe global hunger crisis caused by climate change and population growth (Food Security Information Network, 2018). To increase food production, many researchers dedicate their efforts on developing automated systems to monitor

The authors are solely responsible for the content of this meeting presentation. The presentation does not necessarily reflect the official position of the American Society of Agricultural and Biological Engineers (ASABE), and its printing and distribution does not constitute an endorsement of views which may be expressed. Meeting presentations are not subject to the formal peer review process by ASABE editorial committees; therefore, they are not to be presented as refereed publications. Publish your paper in our journal after successfully completing the peer review process. See www.asabe.org/JournalSubmission for details. Citation of this work should state that it is from an ASABE meeting paper. EXAMPLE: Author's Last Name, Initials. 2019. Title of presentation. ASABE Paper No. ---. St. Joseph, MI.: ASABE. For information about securing permission to reprint or reproduce a meeting presentation, please contact ASABE at www.asabe.org/permissions (2950 Niles Road, St. Joseph, MI 49085-9659 USA).

and control plant growth. Two of the several techniques proffered are phenotyping and breeding. Phenotyping is used to collect quantitative information on plant characteristics and growth parameters while breeding utilizes the analytical results to help plants easily adapt to resource-limited environments. However, traditional phenotyping methods are time consuming, labor intensive and prone to human error. With the rapid development of image processing and artificial intelligence techniques, computer vision-based agricultural robots or unmanned aerial vehicles are being developed to efficiently perform such methods.

Traditional image processing methods for fruit detection use color information, geometric features or texture to classify fruits or leaves (Zhao, et al., 2005). However, hand-crafted feature extractors are easily affected by environmental factors such as various lighting or similarities between fruit and background. To overcome this issue, traditional image processing methods for fruit detection are being replaced with deep learning techniques. In deep learning, fruit detection is done using convolutional neural network- (CNN) based object detectors. Faster R-CNN, one of the most popular object detectors, is a single, unified network. The region proposal networks (RPN) are used to serve as the focus of ‘attention’ of the network (Ren et al., 2015). Similarly, object detection by You Only Look Once (YOLO) treats images as a regression problem to spatially separate bounding boxes and associate class probabilities (Redmon et al., 2016; Redmon and Farhadi, 2017). Both deep learning networks are able to detect and locate objects simultaneously in an image. One of the works that uses deep learning for fruit detection is by Sa et al. (2016). They used Faster R-CNN to detect fruits in color and near-infrared images, with an F_1 score of 0.838. Similarly, Bargoti and Underwood (2016) also used Faster R-CNN to detect apples and mangoes, with an F_1 score above 0.9. To compare the performance between Faster R-CNN and YOLO, Underwood et al. (2018) did fruit detection using the YOLO model, with an F_1 score of 0.951.

Together with automated fruit detection, tracking and localization functions can also be performed. Through this process, farmers can achieve higher productivity, predict yield and improve crop quality. Moonrinta et al. (2010) used monocular cameras and speeded up robust feature (SURF) points to detect the fruit regions and support vector machine (SVM) for feature point classification, with a hit rate of 91.37%. By blob tracking, they performed fruit tracking and structure from motion (SfM) to obtain 3D reconstructions. Meanwhile, Liu et al. (2018) presented a novel pipeline for counting, tracking and localizing fruits through fruit and non-fruit pixels segmentation, across frames tracking and SfM-based 3D localization.

The specific objectives of this study are to collect images and video streams by RGB-D camera for training the object detector model, develop a fruit tracking algorithm, and to measure and validate individual fruit traits. To meet these challenges, an imaging framework for fruits detection, localization and measurement was presented. By scanning through the crops row by row in a greenhouse, fruit counts, individual fruit ripening stages and 2D spatial localization maps are obtained. For fruit detection, a deep learning-based object detector based on YOLOv2 was trained. A finite state machine and SfM-based tracking method is proposed. This work can be applied not only for the tested fruits in this study, but also for other kinds of fruits by simply retraining the object detector models.

Materials and Methods

Image and video data collection

Sample images and videos were collected from an experimental greenhouse located in Chin-Yong farm, Hsinchu, Taiwan. The imaging equipment used includes tripod, tripod dolly, Feiyu SPG 3-axis handheld stabilizer and Intel RealSense D435 depth camera, as shown in Figure 1.



Figure 1. Experimental setup for video stream acquisition

Both RGB and depth images were collected using the D435 depth camera with resolutions of 1920 x 1080 and 1280 x 720, respectively. By prior testing, the calibrated minimal depth resolution is 0.256 m to set the camera 0.3 m far from the targets. Fifteen video streams were collected by the camera where a total of five rows were used for collection, and each row was recorded three times. Sample RGB and depth images are shown in Figure 2.

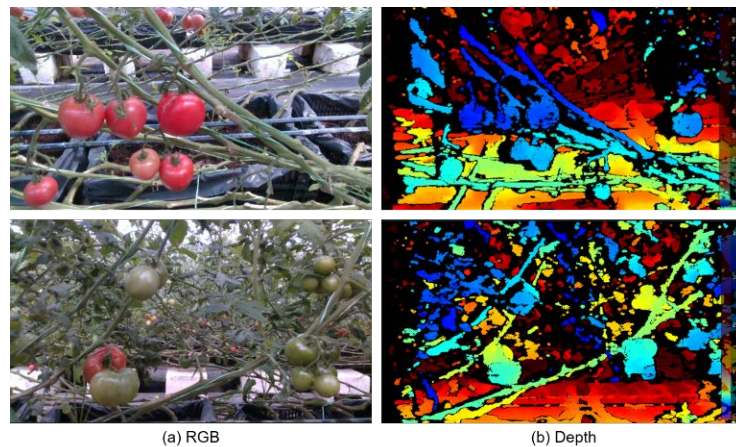


Figure 2. (a) RGB and (b) depth image samples collected from Chin-Yong farm

Algorithm overview

The flowchart of the algorithm is shown in Figure 3 with two main components: detection and tracking. First, the YOLOv2 fruit detector is trained by several training images collected from Chin-Yong farm. The detector then detects fruits in each RGB frame from video streams. Second, a fruit tracking algorithm is performed on each frame using the SfM tracking method. The input RGB frames are filtered using the depth information to keep only the front side crops, the measured distance is 0.3 m ~ 1.0 m. Meanwhile, tracking was done in two parts: online tracking and offline tracking. Online tracking is used to identify fruits between frames and handle the occlusion problem, while offline tracking eliminates false alarms by the detector or depth information. Finally, total fruit count, individual fruit size, and 2D spatial distribution map are obtained by analyzing the tracking results.

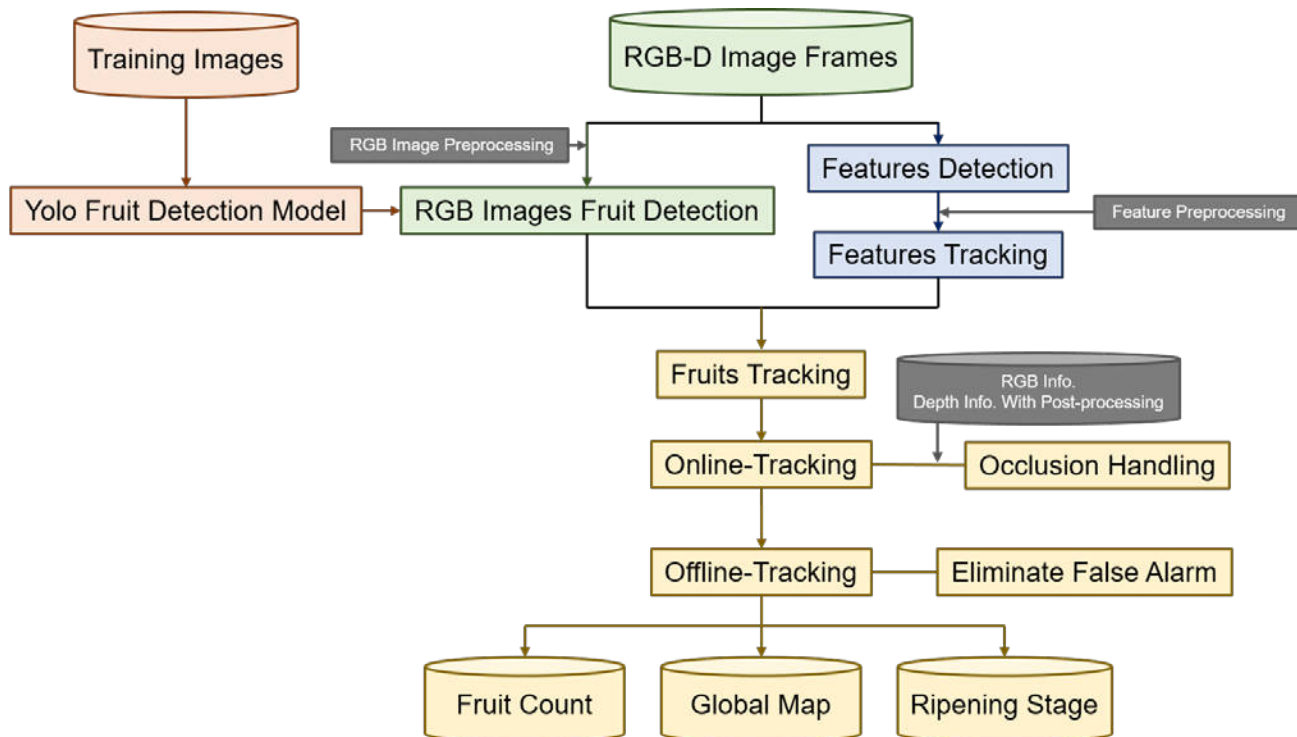


Figure 3. Algorithm flowchart

Fruit detection

The YOLOv2 model, with input image sizes of 416 x 416, was used to detect fruits in an image. It splits the input image into a few grid cells and regresses the object bounding boxes coordinates in each cell. To prepare the training data, Labellmg software (Lin, 2015) was used for annotation. Eight videos were randomly chosen among the 15 videos for sampling one training image per twenty frames. Among the training image set items, 100 validation data were randomly chosen. In some images, the fruits' occlusion level varies. Therefore, only fruits with above 50% visible area were labeled to train the 50%-visible model, while 100% visible were picked out to train the 100%-visible model. The numbers of training and validation images are shown in Table 1.

Table 1. Number of training and validation images

	Training Data	Validation Data
50% - visible model	340 tomato images	100 tomato images
100% - visible model	340 tomato images	100 tomato images

For model testing, 100 images, differing from the training data, were sampled from the 15 videos. Three model evaluation metrics were used: hit rate, miss rate and false alarm rate, calculated using Equations 1, 2, and 3:

$$\text{Hit rate} = \frac{\sum TP}{\sum \text{Actual positive}} \quad (1)$$

$$\text{Miss rate} = \frac{\sum FN}{\sum \text{Actual positive}} \quad (2)$$

$$\text{False alarm rate} = \frac{\sum FP}{\sum \text{Actual negative}} \quad (3)$$

The detected bounding boxes that properly fit the ground truth bounding boxes by manually verification are considered as true positive (TP) detection. Missed bounding boxes are considered false negative (FN) and the overly detected bounding boxes are considered false positive (FP).

Fruit tracking

To identify individual fruits between frames, finite state machine (FSM) and SfM were used, as shown in Figure 4.

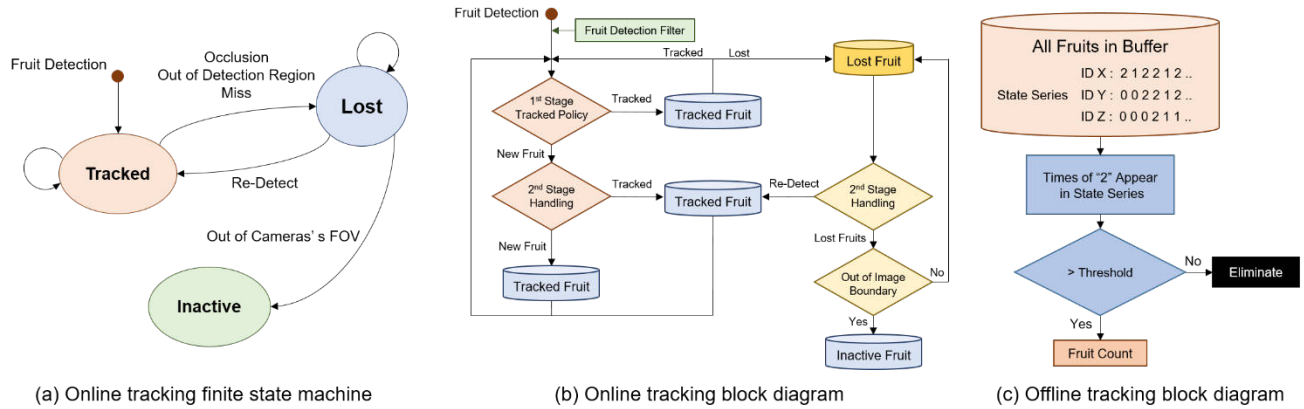


Figure 4. (a) Finite state machine mechanism, (b) online tracking block diagram and (c) offline tracking block diagram

Three states are included in the FSM: tracked, lost and inactive. The finite state machine mechanism is shown in Figure 4(a). As long as a fruit is detected, it is tracked. For fruits missed by the object detector or out of detected region, it will be set into the lost state. Finally, fruits in the lost state will be set as inactive if it is out of the camera's field of view, and set as tracked if it is re-detected by the detector. The fruit tracking algorithm contains online and offline tracking, as shown in Figure 4(b) and Figure 4(c). Online tracking is used for identifying each fruit between frames, while offline tracking is used for denoising by eliminating false alarms in the tracking sequences. To only take the front side fruits into account, the images were preprocessed by applying binary thresholding by depth information as shown in Equation 4. This allows the scenery far from the camera to appear darker, where α is in the range of 0 and 1:

$$I_{ij} = \begin{cases} I_{ij}, & \text{lower bound} \leq D_{ij} \leq \text{upper bound} \\ \alpha I_{ij}, & D_{ij} \geq \text{lower bound} \text{ or } D_{ij} \leq \text{upper bound} \end{cases} \quad (4)$$

where I_{ij} is the pixel value of image $I(i, j)$ and D_{ij} is the depth value of image $I(i, j)$. In this work, the lower bound is equal to 0.3 m, while the upper bound is equal to 1.0 m; the distance of the camera setting in front of the crop falls under the set

threshold. On the other hand, α is 0, 1/4, 1/5 to show how the background affects the results by parameter testing. After detecting the fruits from the preprocessed frame, two criteria were set to filter out the detection false alarm: leaf or non-front side fruits. First, the median distance of the detected fruit's bounding box should be in the range of the lower bound and upper bound as defined above. Second, the standard deviation of the distance of a detected fruit's bounding box should be smaller than a threshold such that the object has a smooth circular shape.

SfM is performed in the online tracking. In the first stage tracked policy, as shown in Figure 4(b), tracking features proposed by Shi and Tomasi (1993) are used to detect feature points in each frame, while optical flow is used to match them between frames (Lucas and Kanade, 1981). Since the system only detects the frontal row, feature points which are too far from the camera were eliminated from the matching set. In addition, object movement between frames should be seen as rigid transformations. Therefore, fruit movements between frames can be approximated as a rigid transformation between the matching feature points. Finally, it predicts the position of each individual fruit in the next frame. The object detector detects the fruits in each frame. By calculating the $L2$ distance between the predicted position and the detected position, a distance threshold is set to determine which ID the detected fruits should be. To formulate, suppose $P_{t,tr} = \{p_t^1, \dots, p_t^{M_t}\}$ denotes the positions of all the M_t objects in tracked state in the t -th frame, then:

$$\exists x \in M_t, Distance_i = \|p_t^x - H \times p_{t-1}^i\|_2, \forall i \in M_{t-1} \quad (5)$$

$$\hat{i} = \underset{i}{\operatorname{argmin}} Distance_i \quad (6)$$

$$\text{if } Distance_{\hat{i}} < thresh, \quad \text{then } x = \hat{i} \\ \text{otherwise } x = 0 \quad (7)$$

where H is the rigid transformation matrix. The distance threshold is set dynamically based on the size of the object. In addition, if the $L2$ distance between two detected object is too close, both distance thresholds are divided by two to decrease the probability of false ID matching. The distance threshold, intersection over union (IoU) values of matched boxes and median depth difference were also taken into account. The IoU value is computed using Equation 8:

$$IoU = \frac{\operatorname{area}(B_D \cap B_P)}{\operatorname{area}(B_D \cup B_P)} \quad (8)$$

where B_D is the bounding box coordinates of the detected fruit and B_P is the bounding box coordinates of the predicted fruit from the former frame. For fruits with $x = 0$, it means that those fruits did not match with any predicted fruit from the last frame. However, they may be some re-detected fruits from the lost state. The second stage occlusion handling is then performed. Similar to the first stage, predicted fruit positions were calculated throughout each frame. Distance, IoU and cumulated fruit median depth difference thresholds were also considered for identification.

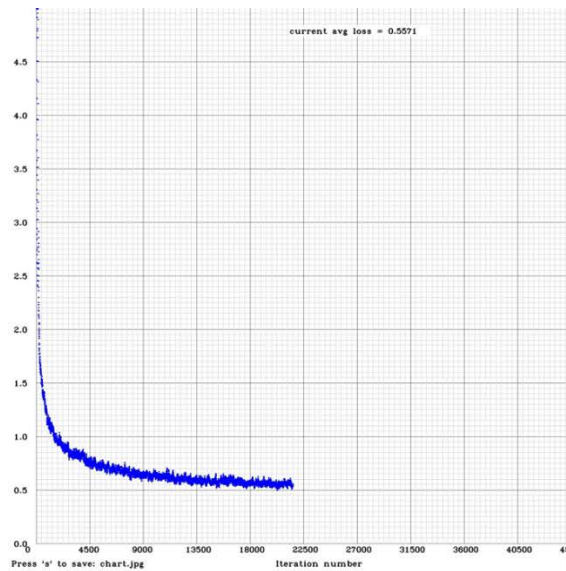
Finally, an offline tracking algorithm is performed on the online tracking results, as shown in Figure 4(c). If the instances of the tracked state appear less than a threshold in the whole state sequence by binary thresholding, then it is defined as a false alarm. The threshold is highly dependent on the false alarm rate and hit rate of the object detector. If the false alarm rate is low and the hit rate is high, then the threshold should be lower, and vice versa. In this work, the detector has a hit rate of 84.98% and a false alarm rate of 15.02%; the threshold was set to 2 by prior parameter tuning.

After tracking, fruit size, maturity stage and 2D distribution map were evaluated. Individual relative fruit sizes were calculated by multiplying the width and height from the YOLO detected bounding boxes; the maturity stage is calculated by GraphCut algorithm (Rother et al., 2004); the color thresholding and distribution map were calculated by inverse rigid transformation.

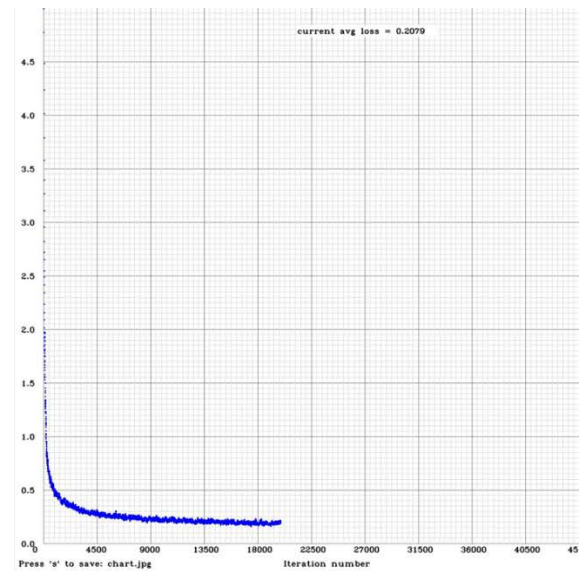
Results and Discussion

Model tuning and evaluation

While training, each kind of model goes through 45000 iterations in maximum. In the first 1000 iterations, a model for each set of 100 iterations was saved. In the last 44000 iterations, a model for each set of 1000 iterations was saved. Therefore, there were a maximum of 54 models in total in each kind of model. The loss during training was plotted, as shown in Figure 5. The figure showed that about 20000 iterations were enough for the 50%-visible model to converge, while it took less iterations to converge for the 100%-visible model. Moreover, the average training loss of 100%-visible model was lower. In testing the model on the raw data, hit rates, false alarm rates and miss rates were used to evaluate the models performance.



(a) Training loss of 50%-visible model



(b) Training loss of 100%-visible model

Figure 5. Training loss of (a) 50%-visible and (b) 100%-visible model

From the results obtained, the 50%-visible and 100%-visible model has a hit rate of 84.98% and 67.91%, respectively. The 100%-visible model was determined overfitting as it had the poor performance on testing data and had the better performance on training data. Therefore, the 50%-visible model was then chosen for the tracking algorithm with a higher hit rate. Sample images of the testing results under the best iteration are shown in Figure 6. Two images on the upper row are the same as sampled from the video No. 10 and the two images on the lower row are sampled from video No. 5.

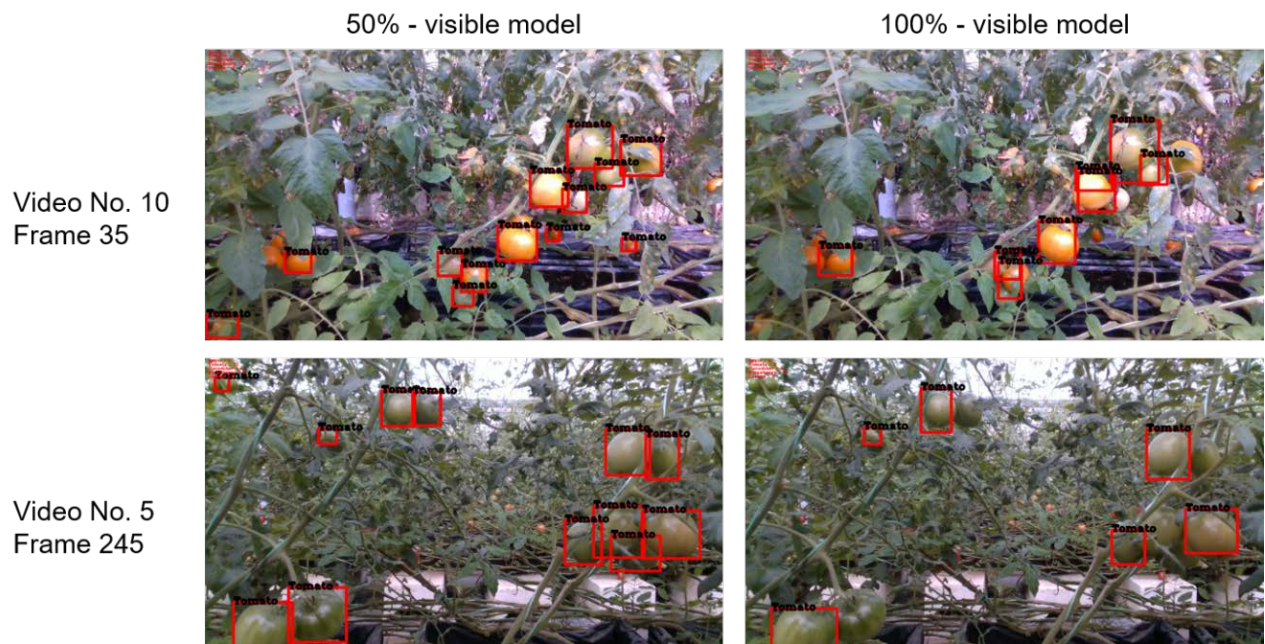


Figure 6. Typical results of testing samples

Analysis of fruit tracking results

The results of the total fruit count and absolute relative fruit count error for each video sorted with the actual fruit counts are shown in Figure 7 with the absolute relative fruit count error calculated using Equation 10.

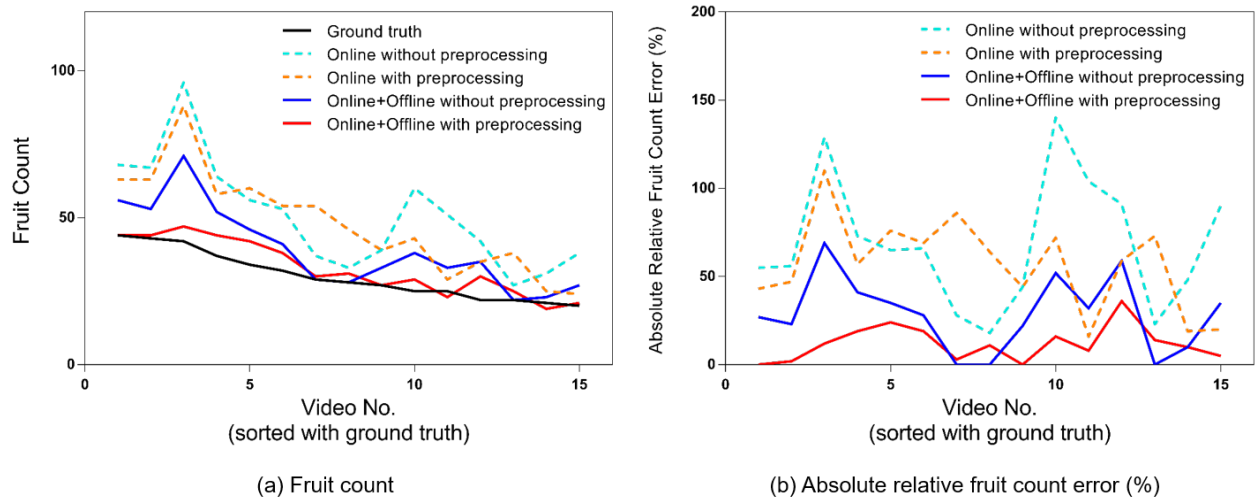


Figure 7. (a) Total fruit count and (b) absolute relative fruit count error results

$$\text{Absolute relative fruit count error (\%)} = \frac{|\text{Automatic count} - \text{Actual count}|}{\text{Actual count}} \times 100 \quad (10)$$

It was observed that the algorithm with preprocessing, including online and offline tracking, can reach the least average absolute relative fruit count error of 12%. With online tracking alone, the counting results were highly dependent on the detection results and were affected by depth noises generated by the camera. In addition, due to the irregular environment, the performance of feature matching varied under different circumstances. Fortunately, the first two issues were solved by image and feature preprocessing and fruit detection filter. Without the image preprocessing step, the average absolute relative fruit count error increased from 12% to 29%, as graphically compared in Figure 8. It can obtain the best result when α is 0.

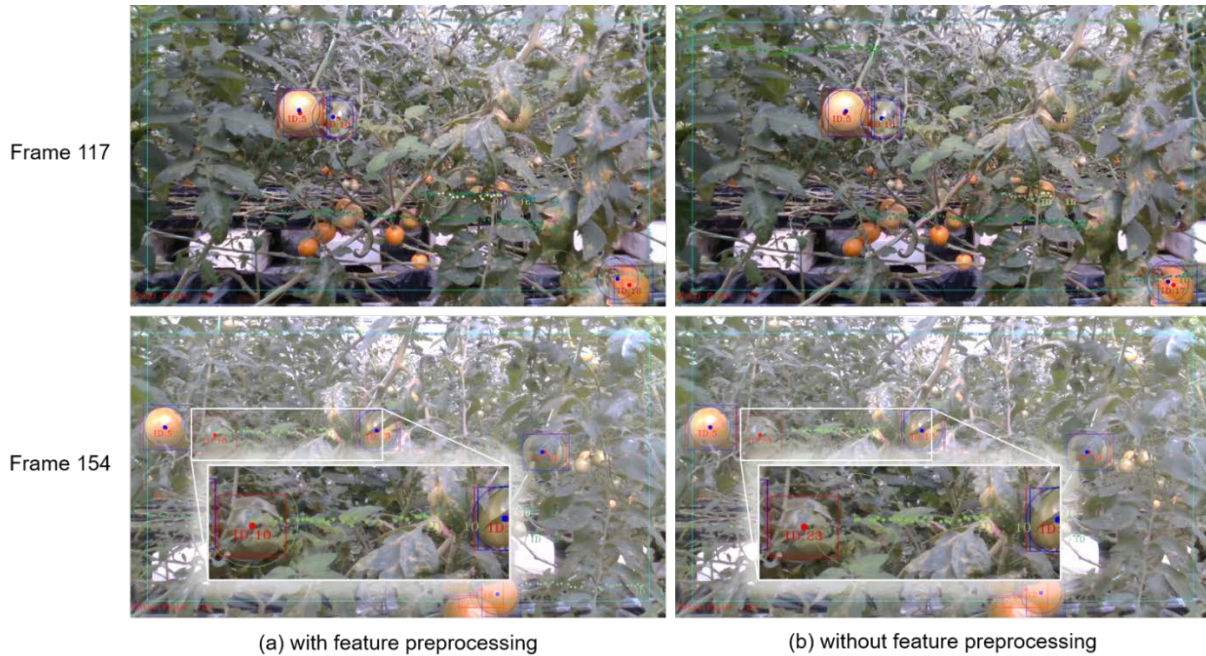


Figure 8. Fruit counting results comparison (a) with and (b) without feature preprocessing (Fruit ID: 10)

In frame 117, the fruit with ID 10 was detected. However, from frame 117 through 154, the fruit was occluded by the leaves and missed by the detector. Through these frames, light green dots were used to represent the predicted fruit position. If the feature preprocessing was not performed, it was noticeable that the light green dots series cannot follow the movement of the fruits and named it with a wrong ID of 23. In order to solve the feature matching bias problem, rigid transformation was applied. Compared to the full affine transformation, rigid transformation has only four degrees of freedom. Therefore, it is more unlikely to generate erratic frame transformation matrices, which are sometimes caused by camera distortions, rocky surfaces or blurred images. Without the rigid transformation applied, the average absolute relative fruit count error increased from 12% to 25%.

In online tracking, the results of the first stage and second stage are shown in Figure 9(a).

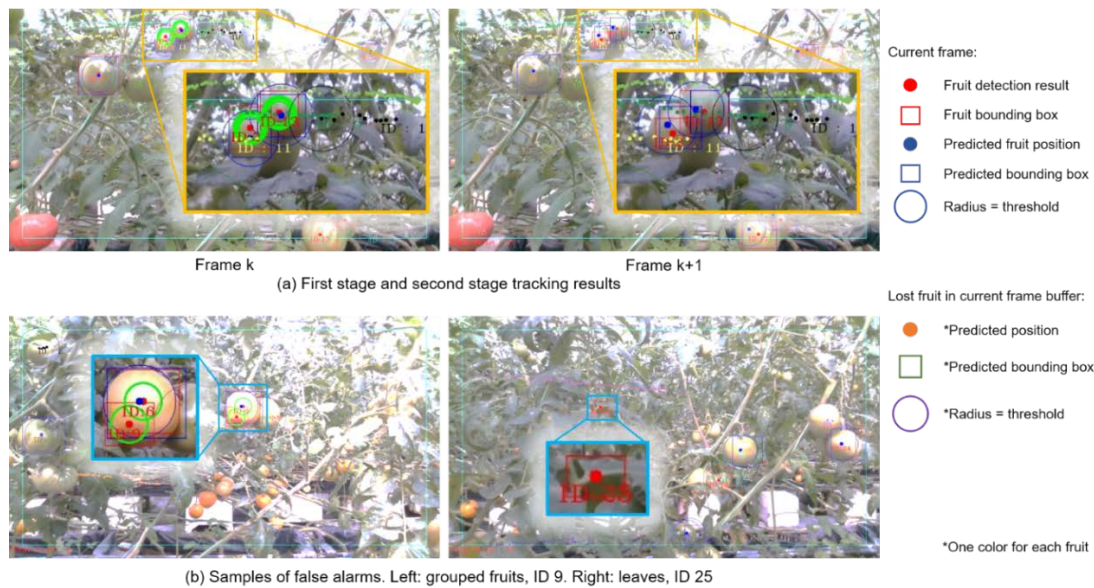


Figure 9. First and second stage fruit tracking results

The predicted positions were calculated using a rigid transformation matrix. The detected results were from the YOLO detector. If the centers of the detected fruits are inside the blue circle, it means that the distance between the detected fruit and predicted fruit is smaller than the threshold; thus, the detected fruit then belongs to the fruit appearing in the last frame. The threshold was set based on the fruit size calculated from the YOLO detector, which is general for different fruit sizes. The series of dots, with different colors, were the lost fruits in the buffer. Whenever a fruit was missed by the detector, its position was stored for second stage processing. Moreover, there were two bright green circles in Figure 9(a), Frame k, which means that the distance between fruits with ID 2 and 3 were too close; their distance thresholds were then smaller. These results show that most false alarms occur due to grouped fruits and leaves. With grouped fruits, it is harder to clarify where and how many the fruits are, as some leaves sometimes look like fruits. In offline tracking, samples of eliminated fruits are shown in Figure 9(b).

2D fruit localization and measurements

Results of individual ripening stage measurements are shown in Figure 10.

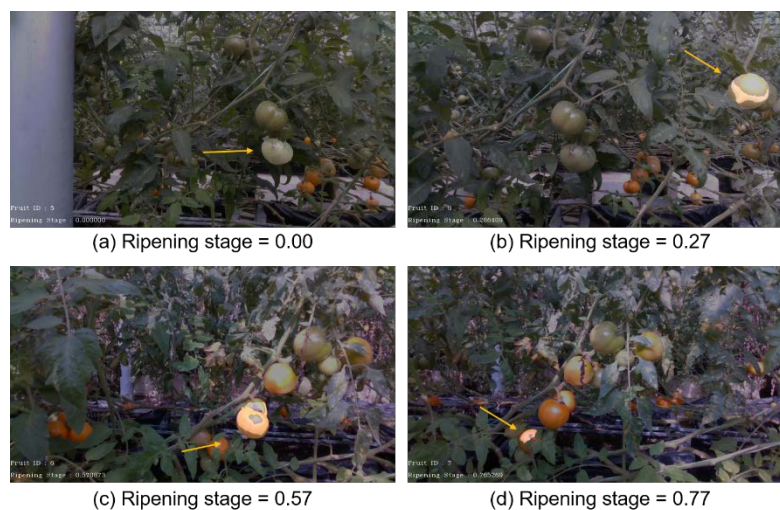


Figure 10. Typical examples of determining individual fruit ripening stage

By using the GraphCut algorithm, fruits were segmented from the background. By thresholding, each pixel defined as a fruit was determined as ripe or not ripe. If it was ripe, the pixel values were higher. However, the detector may have bias on positions of detected bounding boxes due to the occlusion or grouping issue. Therefore, the GraphCut algorithm was only used on the visible parts, like in Figure 10(d). Besides from detector bias, the results were also affected by various illumination even when the experiments were performed inside the greenhouse.

The 2D fruit localization map result is shown in Figure 11.

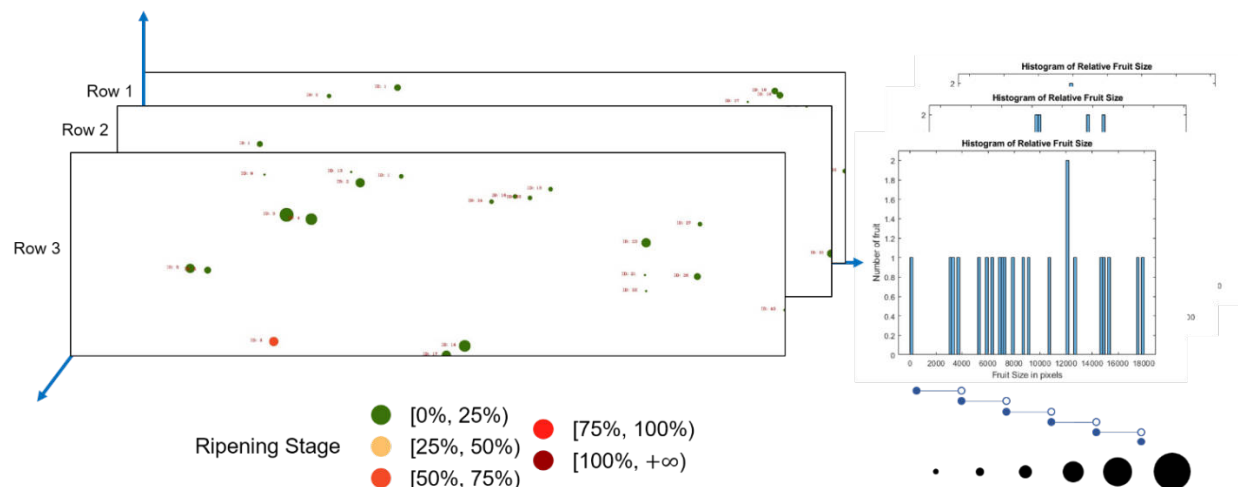


Figure 11. 2D fruit localization map showing spatial phenotyping information

Ripening stage and size of the fruits were represented using dots with different colors and sizes. In theory, tomatoes have 6 growth stages. During the third stage, tomatoes change color from green to red and the pulps become softer from the fifth stage to the sixth stage. Using this information, the ripening stages and relative fruit size were divided into the 5th and 6th stage. The colors chosen were closed to the ground truth color. The green and light orange colors signify the unripen stage, while the orange, red and dark red color signify the gradual ripening stage. Fruit sizes are measured in regard to relative pixel level. In the histogram, the sizes are shown evenly distributed. Moreover, the size of the fruit cannot directly describe its ripening stage.

The information collected by the algorithm can be used by farmers to plan the storage and labor needs before harvesting. For instance, the area with more and bigger fruits should have more manpower for harvesting. On the other hand, if the experiments were done per day, the algorithm can be used to analyze the spatial and temporal information. By comparing the results of the same rows during a specific time, fruits growth measurement can be done. By scanning each row in the greenhouse, the spatial distribution can also be determined. Therefore, farmers can adjust their farming strategies to improve their profit.

Conclusion

This paper presents a method for counting fruits, recording individual fruit sizes, ripening stages and 2D fruit distribution mapping using an RGB-D imaging system. Based on the detection results, the 50%-visible model outperforms the 100%-visible model, with a higher hit rate of 84.98% compared with 67.91%. In the fruit tracking algorithm, the lowest absolute relative fruit count error is 12% with image and feature points preprocessing, rigid transformation of feature points and offline tracking. The experimental results show that the 2D fruit localization map may provide farmers with individual or grouped fruit growth levels, help determine the best environment for growing the fruits and plan different harvesting methods. The proposed framework has demonstrated its efficacy in obtaining satisfactory phenotyping information that is useful for production management as well as its potential utilization in robotic operations. Currently, few errors still occur due to the detector false alarms, inaccurate feature matching algorithm and the blurred images caused by image acquisition errors. As future work, it is recommended to add more training data to increase the accuracy of the detector and gather more data for analysis.

Acknowledgements

The authors would like to thank the staff members of Chin-Yong Farm in Hsinchu, Taiwan, for allowing us to collect the image samples from their site.

References

- Bargoti, S., & Underwood, J. (2016). Deep Fruit Detection in Orchards. *arXiv preprint arXiv:1610.03677*.
- Food Security Information Network. (2018). 2018 Global Report on Food Crises. World Food Programme.
- Lin, T. T. (2015). LabelImg. Retrieved from: <https://github.com/tzutalin/labelImg>
- Liu, X., Chen, S. W., Aditya, S., Sivakumar, N., Dcunha, S., Qu, C., & Kumar, V. (2018). Robust fruit counting: Combining deep learning, tracking, and structure from motion. *Proc. IEEE/RSJ Int. Conf. on Intell. Robots Syst.*, 2018, pp. 1045–1052.
- Lucas, B. D., & Kanade, T. (1981). An iterative image registration technique with an application to stereo vision. *Proc. Int. Joint Conf. on Artificial Intelligence*, 1981, pp. 674-679.

- Moonrinda, J., Chaivivatrakul, S., Dailey, M. N., & Ekpanyapong, M. (2010). Fruit detection, tracking, and 3D reconstruction for crop mapping and yield estimation. *IEEE, 11th Int. Conf. on Control Automation Robotics & Vision*, pp. 1181-1186.
- Redmon, J., & Farhadi, A. (2017). YOLO9000: better, faster, stronger. *arXiv Preprint, arXiv:1612.08242, 2016*
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proc. IEEE Int. Conf. CVPR*, 2016, pp. 779–788.
- Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. *Proc. 28th Int. Conf. Neural Inf. Process. Syst.*, pp. 91–99.
- Rother, C., Kolmogorov, V., & Blake, A. (2004, August). Grabcut: Interactive foreground extraction using iterated graph cuts. *ACM transactions on graphics*, vol. 23, no. 3, pp. 309-314.
- Sa, I., Ge, Z., Dayoub, F., Upcroft, B., Perez, T., & McCool, C. (2016). Deepfruits: A fruit detection system using deep neural networks. *Sensors*, vol. 16, no. 8, p. 1222.
- Shi, J., & Tomasi, C. (1994). Good features to track. *Proc. IEEE Computer Society Conf. on Computer Vision and Pattern Recognition*, pp. 593-600.
- Underwood, J. P., Rahman, M. M., Robson, A., Walsh, K. B., Koirala, A., & Wang, Z. (2018). Fruit load estimation in mango orchards—a method comparison. *ICRA 2018 workshop on robotic vision and action in agriculture*. Brisbane, Australia.
- Zhao, J., Tow, J., Katupitiya, J. (2005). On-tree fruit recognition using texture properties and color data. *Int. Conf. on Intelligent Robots and Systems*. Edmonton Canada, p. 263-268.